Advanced Data Management
Mock Exam v3 (Chapters 1–6)

Same structure as Mock v2 and the midterm, new database and new questions: 8 quick checks, 3 problem situations, one real plan, 2 of 3 tickets. Solutions in the separate file.

Advanced Data Management

2026-11-01 100 points 2 hours

Instructions

- **Allowed**: one A4 sheet of notes (recto verso) and the formula sheet on the last page.
- **Show your reasoning.** A wrong final number with a correct method earns most of the points; a correct number without a method earns little. Write the formula, then the numbers with units, then one sentence that quotes your number.
- Suggested timing: Part A 25 min, Part B 35 min, Part C 35 min, Part D 20 min, check 5 min.

Concept: The database: `citycab`, PostgreSQL 16, 8 KB pages

A ride-hailing company. Statistics are fresh unless a question says otherwise.

Table	Rows	Pages	Rows/page	Notes
`rides`	2,000,000	16,667	120	one row per ride; columns `id`, `rider_id`, `driver_id`, `zone_id`, `pickup_city`, `ride_ts`, `fare`, `status`, 40 bytes of column data per row
`riders`	2,000,000	20,000	100	passengers; `riders_pkey` on `id`, 3 levels
`drivers`	20,000	160	125	`drivers_pkey` on `id`
`zones`	50	1	50	a zone lies inside exactly one city

Indexes on `rides`: `rides_pkey` on `id` (fanout 200), `rides_zone_idx` on `zone_id` (3 levels), `rides_ts_idx` on `ride_ts` (3 levels, 300 entries per leaf). From `pg_stats` on `rides`: `ride_ts` covers 365 days uniformly, correlation 0.99 (rides are appended in time order); `zone_id` has 50 distinct values, MCV `1`: 0.30, `7`: 0.02, correlation 0.003; `pickup_city` has 12 distinct values, MCV `'Casablanca'`: 0.45, `'Rabat'`: 0.10; `status` MCV `'done'`: 0.95, `'cancelled'`: 0.04, `'disputed'`: 0.01; `driver_id` 20,000 distinct, no MCV list; `fare` uniform between 2 and 52, histogram of 10

equi-depth buckets with bounds $2, 7, 12, \ldots, 52$. Cost constants are the defaults: `seq_page_cost` 1, `random_page_cost` 4, `cpu_tuple_cost` 0.01, `cpu_index_tuple_cost` 0.005, `cpu_operator_cost` 0.0025. `work_mem` is 4 MB (512 pages) unless stated.

Part	Type	What you do	Points
A	Quick checks	8 multiple-choice questions, one-line justification	24
B	Problem situations	3 decisions, each built on a short calculation	30
C	Read a real plan	annotate a real `EXPLAIN (ANALYZE, BUFFERS)` output	26
D	Discussion	choose 2 of 3 tickets, half a page each	20
		Total	100

Part A: Quick checks, circle one and justify in one line (24 pts)

Three points each: one for the letter, two for the justification (a number, a formula or one sentence).

A1. A developer runs `SELECT fare FROM rides WHERE drvier_id = 7;` (a typo in the column name). Which component rejects it, and what does it consult to do so?
(a) the parser, the grammar (b) the binder, the catalog (c) the planner, the statistics (d) the executor, the heap file

A2. `rides` has 40 bytes of column data per row, a 24-byte tuple header and a 4-byte slot; the page header is 24 bytes. Rows per page?
(a) 204 (b) 128 (c) 120 (d) 2,000,000

A3. `rides_pkey` indexes 2,000,000 ids with a fanout of 200. Page reads for one lookup by `id`, heap page included?
(a) 3 (b) 4 (c) 21 (d) 22

A4. The login screen runs `WHERE lower(email) = 'x@y.ma'` on `drivers`. Which index makes it an index lookup?
(a) a B-tree on `email` (b) a B-tree on `lower(email)` (c) a hash index on `email` (d) a BRIN index on `email`

A5. Which of these nodes cannot produce its first output row before it has consumed its whole input?
(a) Seq Scan with a `Filter` (b) Nested Loop (c) HashAggregate (d) Merge Join

A6. `rides` (16,667 pages) is sorted with $M = 130$ buffer pages. Total I/O (read + write, every pass)?
(a) 16,667 (b) 33,334 (c) 66,668 (d) 100,002

A7. Estimated rows for `WHERE status = 'disputed' AND zone_id = 1`

on `rides`?
(a) about 6,000 (b) about 20,000 (c) about 600,000 (d) about 620,000

A8. PostgreSQL's clock-sweep hand reaches an unpinned frame whose `usage_count` is 3. What happens?
(a) the page is evicted (b) the count becomes 2 and the hand moves on (c) the count becomes 5, the page was hit (d) the frame is pinned until the count reaches 0

Part B: Problem situations, calculate then decide (30 pts)

Use the formula sheet. Each situation ends with a decision: answer it in one or two sentences that use your numbers.

Problem: B1 "An index on a 2 % filter never helps, delete it" [10 pts]

The operations dashboard runs every minute:

```
SELECT zone_id, count(*) FROM rides
WHERE ride_ts >= now() - interval '7 days' GROUP BY zone_id;
```

A colleague wants to drop `rides_ts_idx`: "the rule says an index only wins below $1/(4r)$, and this filter keeps about 2 % of the table." Recall: `ride_ts` is uniform over 365 days with correlation 0.99; the index has 3 levels and 300 entries per leaf.

(a) Selectivity of the predicate, estimated rows, and the number of heap pages those rows occupy if they are stored contiguously. [3 pts]
(b) Cost of the Seq Scan with its filter (three terms). Cost of the Index Scan when the rows are contiguous: descent, leaf pages, heap pages, per-row CPU. Which wins, by what factor? [4 pts]
(c) Now suppose the correlation were 0.003 instead: expected number of heap pages touched, cost of the Index Scan, and the winner. State the rule $1/(4r)$ for this table and decide: is the colleague right? What is the single word that explains the difference? [3 pts]

Start with: "The predicate keeps $7/365 = \ldots$ of the rows, that is ... rows, which fill ... pages if ..."

Problem: B2 Driver earnings, sorted by driver [10 pts]

The payroll job joins `drivers` (R, 160 pages, 20,000 rows) with `rides` (S, 16,667 pages) on `driver_id`, then sums the fares per driver and must output the result *ordered by* `driver_id`. No index on `rides(driver_id)`. The job gets $M = 64$ buffer pages.

(a) Cost in page I/O of a block nested loop with `drivers` as the outer relation. [3 pts]
(b) Cost of a hash join: does the build side fit in M? Which variant

applies, and what is its cost? [3 pts]

(c) Cost of a sort-merge join: external sort of each input (runs, passes, I/O), then the merge. [2 pts]

(d) The DBA offers $M = 200$ pages. Recompute the three costs and decide which join to run and how to obtain the sorted output cheaply. [2 pts]

Start with: "With $M = 64$ the outer needs $\lceil 160/62 \rceil = \ldots$ chunks, so ..."

Problem: B3 "The morning after the audit" [10 pts]

The production server has `shared_buffers = 1GB` (131,072 frames). During the day the OLTP working set is 20,000 pages and the measured hit ratio is 99.5 %; a hit costs 0.1 μs, a miss 100 μs (SSD). Every night an audit job does one sequential scan of `rides_archive`, 400,000 pages, and every morning the first minutes are slow.

(a) Effective access time at 99.5 %; how many times slower than RAM? If the morning hit ratio drops to 90 %, new EAT and the slowdown factor against the daytime value. [3 pts]

(b) Under plain LRU, how many pages of the working set survive the scan, and what does the morning cost in reload time? Explain the mechanism in two sentences. [3 pts]

(c) PostgreSQL: is this scan ringed? Give the threshold, the ring size, and say whether the working set survives and what the ring costs the scan itself. [2 pts]

(d) The DBA proposes `shared_buffers = 48GB` on the 64 GB server "so that the archive fits". Two sentences: what the scan gains, and what you measure before deciding. [2 pts]

Start with: "EAT $= h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}} = \ldots$"

Part C: Read a real plan (26 pts)

Problem: Ticket OPS-44 "The Rabat zone-7 dashboard takes two seconds" [26 pts]

The city team's dashboard lists the riders with the most rides in zone 7 of Rabat. Below is the real output of `EXPLAIN (ANALYZE, BUFFERS)` on `citycab` (parallelism off). Zone 7 lies inside Rabat.

```
SELECT p.name, count(*) AS n, sum(r.fare) AS spent
FROM rides r JOIN riders p ON p.id = r.rider_id
WHERE r.zone_id = 7 AND r.pickup_city = 'Rabat'
GROUP BY p.name ORDER BY n DESC LIMIT 20;
```

```
Limit  (cost=34318.42..34318.47 rows=20)
       (actual time=1948.2..1948.3 rows=20 loops=1)
  Buffers: shared hit=136593 read=38600
  ->  Sort  (cost=34318.42..34328.42 rows=4000)
            (actual time=1948.1..1948.2 rows=20 loops=1)
        Sort Key: (count(*)) DESC
```

```
        Sort Method: top-N heapsort  Memory: 27kB
        ->  HashAggregate  (cost=34168.00..34208.00 rows=4000)
                  (actual time=1930.6..1939.8 rows=31207 loops=1)
              Group Key: p.name
              ->  Nested Loop  (cost=848.62..34148.00 rows=4000)
                        (actual time=12.1..1890.4 rows=40000 loops=1)
                    ->  Bitmap Heap Scan on rides r
                          (cost=848.20..17832.60 rows=4000)
                          (actual time=11.9..310.2 rows=40000 loops=1)
                          Recheck Cond: (zone_id = 7)
                          Filter: (pickup_city = 'Rabat'::text)
                          Rows Removed by Filter: 0
                          Heap Blocks: exact=15190
                          Buffers: shared hit=15193
                          ->  Bitmap Index Scan on rides_zone_idx
                                (cost=0.00..847.20 rows=40000)
                                (actual time=9.4..9.4 rows=40000 loops=1)
                                Index Cond: (zone_id = 7)
                                Buffers: shared hit=3
                    ->  Index Scan using riders_pkey on riders p
                          (cost=0.43..4.07 rows=1)
                          (actual time=0.038..0.039 rows=1 loops=40000)
                          Index Cond: (id = r.rider_id)
                          Buffers: shared hit=121400 read=38600
Planning Time: 0.4 ms
Execution Time: 1950.1 ms
```

1. Draw the plan as a tree (7 nodes). For each node say whether it must consume all its input before producing its first row, with a few words of why. Why does the `LIMIT 20` not make the query fast? [5 pts]
2. Where do `rows=40000` on the Bitmap Index Scan and `rows=4000` on the Bitmap Heap Scan come from? Give the q-error of the Heap Scan, name the estimation failure, say whether `ANALYZE rides` fixes it, and give the command that does, with the estimate it produces. [5 pts]
3. Explain the numbers `Heap Blocks: exact=15190` and `Buffers: shared hit=121400 read=38600` on the inner Index Scan. Where do the two seconds go? Is the overall hit ratio a useful number here? [6 pts]
4. With the correct estimate, which join would the planner choose, with which side as build, and roughly what would it cost in page I/O and in time? Compare with the actual cost of the nested loop. [5 pts]
5. Two proposals: (i) `CREATE INDEX ON rides (zone_id, pickup_city)`; (ii) `CLUSTER rides USING rides_zone_idx`. For each: what changes in the plan lines, in `Heap Blocks`, in the estimate and in the time? Which should the team do first? [5 pts]

Start with: "The Bitmap Index Scan keeps $0.02 \times 2{,}000{,}000 = \ldots$ rows, then the Filter multiplies by $\ldots$ "

Part D: Discussion, choose 2 of the 3 tickets (20 pts)

Ten points each, half a page. A good answer: claim → mechanism → number → consequence. Cross out the ticket you do not answer.

Problem: Ticket D1 "Move `rides` to a column store, analytics are slow" [10 pts]

The analytics team's main query sums `fare` per `zone_id` per day (three of the 25 columns of the full `rides` table) and reads all 16,667 pages. The app, meanwhile, inserts 2,000 rides per second and reads single rides by `id`. A consultant proposes converting the table to a column store. Explain what each workload would gain or lose, with a number for the analytics scan and one for an insert, and propose the architecture you would actually recommend, with the PostgreSQL commands involved. Start with: "A column store reads ... of the bytes for this query, because ..."

Problem: Ticket D2 "Replace clock-sweep with true LRU, LRU is optimal" [10 pts]

A colleague read that LRU has the stack property and concludes PostgreSQL should maintain an exact LRU list for its buffer pool. Explain which policy is actually optimal and why LRU is only an approximation of it, what an exact LRU list costs on a hit and why that matters at 99 % hit ratios, how clock and PostgreSQL's `usage_count` approximate LRU and what they add, and name the workload on which every one of these policies fails and what PostgreSQL does about it. Start with: "LRU is not optimal; the optimal policy evicts ..."

Problem: Ticket D3 "Planning took 30 seconds, set `join_collapse_limit = 1` everywhere" [10 pts]

A nine-table reporting query spent 30 seconds in the planner. A developer wants `join_collapse_limit = 1` server-wide so that PostgreSQL "just joins the tables in the order we wrote them". Explain what the planner is searching (give the number of left-deep orders and the number of subsets System-R dynamic programming evaluates for nine tables), what the setting does and what it risks, why the written order is rarely the best one (use selection pushdown in your argument), and what you would propose instead. Start with: "For nine tables the planner considers ..."

Formula sheet

Topic	Formula
Pages of a table	rows per page $= \lfloor (8192 - 24)/(\text{record} + 4) \rfloor$, record = columns +24 rounded up to 8; pages = $\lceil$rows/rows per page$\rceil$
B$^+$-tree	levels $= \lceil \log_F(\text{entries}) \rceil$; lookup = levels +1 heap page; range = levels $+\lceil k/L \rceil - 1$+ heap pages
Index versus scan	plain index scan wins iff $4sN < B$, i.e. $s < 1/(4r)$; heap pages touched $= B\,(1 - (1 - s)^r)$ when correlation ≈ 0, $\approx sB$ when correlation $\approx \pm 1$
Cost units	Seq Scan $= B + 0.01N + 0.0025\,N$ per predicate; random page 4; index entry 0.005; tuple 0.01
External merge sort	runs $= \lceil B/M \rceil$; merge passes $= \lceil \log_{M-1}(\text{runs}) \rceil$; total I/O $= 2B \times (1 + \text{merge passes})$
Block nested loop (R outer)	$B_R + \lceil B_R/(M-2) \rceil \times B_S$
Index nested loop	$B_R + \lvert R \rvert \times$ (index pages + matching heap pages)
Sort-merge join	$\text{sort}(R) + \text{sort}(S) + B_R + B_S$
Hash join	build fits in memory: $B_R + B_S$; otherwise (Grace): $3(B_R + B_S)$
Selectivity	= value: MCV frequency, else $1/n_{\text{distinct}}$; range: fraction of histogram buckets; AND: $s_1 s_2$; OR: $s_1 + s_2 - s_1 s_2$; join on a key: $\lvert R \rvert \lvert S \rvert / \max(d_R, d_S)$
q-error	$\max(\text{est}/\text{actual}, \text{actual}/\text{est})$
Buffer pool	EAT $= h\,T_{\text{hit}} + (1 - h)\,T_{\text{miss}}$; $h =$ hit/(hit + read); a Seq Scan of a table larger than `shared_buffers`/4 uses a 32-frame ring